

RAPORT

TEMAT 13 - JASKINIA

SZYMON LOTTIG

NR ALBUMU: 155286

LINK DO REPOZYTORIUM: [GITHUB](#)

1. Założenia projektowe i opis ogólny

Opis zadania:

Celem projektu "Jaskinia" (Temat 13) było stworzenie symulacji współbieżnej obsługi ruchu turystycznego w środowisku systemu Linux. Kluczowym założeniem było uniknięcie rozwiązań scentralizowanych – symulacja opiera się na niezależnych procesach komunikujących się za pomocą mechanizmów IPC (Inter-Process Communication).

Główne wyzwania implementacyjne obejmowały:

- Synchronizację dostępu do współdzielonych zasobów (kładka wejściowa jako wąskie gardło, limity miejsc na trasach).
- Obsługę priorytetów i zróżnicowanych typów klientów (zniżki, ograniczenia wiekowe).
- Sterowanie symulacją w czasie rzeczywistym za pomocą sygnałów.

Architektura systemu

System został zaprojektowany w modelu zarządcy i pracowników. Zgodnie z wymaganiami, do uruchamiania procesów wykorzystano funkcje `fork()` oraz `exec()`.

Wszystkie zasoby IPC (pamięć dzielona, semaforey, kolejka komunikatów) są tworzone z flagą uprawnień `0600` (odczyt/zapis tylko dla właściciela), co realizuje wymóg minimalnych praw dostępu. Implementacja obejmuje pełną obsługę błędów funkcji systemowych przy użyciu `perror()`

Opis modułów (plików źródłowych)

Program podzielono na wyspecjalizowane moduły, z których każdy odpowiada za inny aspekt symulacji:

- **Main.cpp (Manager Systemu)**

Pełni rolę inicjatora. Jego zadaniem jest:

1. Utworzenie zasobów systemowych (IPCS).
2. Ustawienie wartości początkowych semaforów i pamięci dzielonej.

3. Uruchomienie procesów potomnych: kasjer, klient (generator) oraz przewodnik przy użyciu funkcji `execlp`.
 4. Obsługa sygnału SIGINT (Ctrl+C), co gwarantuje bezpieczne usunięcie zasobów i zakończenie wszystkich procesów potomnych przed wyjściem.
- **klient.cpp (Generator Ruchu i Logika Turysty)**
Jest to najbardziej złożony moduł, symulujący zachowanie turystów.
 1. Losuje cechy (wiek, typ biletu) i wysyła zgłoszenie do Kasjera przez kolejkę komunikatów.
 2. Implementuje logikę "inteligentnego" oczekiwania na wejście na kładkę (funkcja `czekaj_na_most`), która weryfikuje kierunek ruchu i dostępność miejsca, zapobiegając zakleszczeniom.
 - **kasjer.cpp (System Biletowy)**
Proces działający w pętli nieskończonej. Odbiera komunikaty z kolejki, weryfikuje uprawnienia do zniżek (dzieci < 3 lat darmowe, powroty -50%) i zlicza statystyki finansowe. Obsługuje priorytetyzację wiadomości – bilety typu "powrót/VIP" (`mtype=2`) są pobierane przed zwykłymi zgłoszeniami.
 - **przewodnik.cpp (Monitoring i Status)**
Proces odpowiedzialny za wizualizację stanu jaskini. Cyklicznie odczytuje dane z pamięci dzielonej i wyświetla je w terminalu. Rejestruje obsługę sygnałów SIGUSR1 i SIGUSR2, które służą do dynamicznego zamykania i otwierania tras zwiedzania.
 - **straznik.cpp (Konsola Sterująca)**
Osobny program interaktywny dla użytkownika. Podłącza się do istniejącej pamięci dzielonej, pobiera PID procesów zarządzających i pozwala na wysyłanie sygnałów sterujących (zamknięcie trasy, ewakuacja)
 - **tools.cpp / common.h / cleanup.cpp**
Pliki pomocnicze zawierające definicje struktur (np. `CaveState`, `TicketMessage`), wrappery na funkcje systemowe (upraszczające obsługę semaforów i pamięci) oraz program awaryjny do czyszczenia zasobów w razie błędu krytycznego.

2. Opis implementacji i napotkane problemy

Zrealizowane mechanizmy:

a) Synchronizacja kładki (Wąskie gardło)

Zaimplementowano ruch jednokierunkowy z limitem osób. Procesy turystów weryfikują zmienną `bridge_direction` w pamięci dzielonej w sekcji krytycznej (chronionej semaforem), co zapobiega zakleszczeniom i wymusza ruch wahadłowy.

b) System kolejkowy z priorytetami

Wykorzystano typy wiadomości w kolejkach komunikatów. Bilety powracających klientów mają `mtype=2` i są pobierane przez Kasjera priorytetowo (flaga `IPC_NOWAIT` dla typu 2), przed biletami zwykłymi.

c) Sterowanie zewnętrzne i obsługa sygnałów

Proces Strażnika wysyła sygnały systemowe (`SIGUSR1/2`) do Przewodnika, który zmienia flagi dostępności tras w pamięci dzielonej, dynamicznie blokując wejście nowym turystom.

Napotkane problemy i rozwiązania

- **Wyścig przy logowaniu:** Przy dużej liczbie procesów komunikaty na ekranie mieszały się.
Rozwiązanie: Implementacja funkcji `safe_log` z użyciem dodatkowego semafora jako mutexu dla operacji wejścia/wyjścia.
- **Procesy Zombie i blokada zasobów:** Po przerwaniu symulacji (`Ctrl+C`) zasoby IPC nie były zwalniane.
Rozwiązanie: Obsługa sygnału `SIGINT` w `main.cpp`, która zapewnia poprawne zamknięcie procesów potomnych (`wait`) i usunięcie zasobów. Dodatkowo stworzono awaryjny program `cleanup`.
- **Trwała blokada trasy po ponownym otwarciu:** Po zamknięciu trasy przez Strażnika, a następnie jej ponownym otwarciu, nowe procesy nie mogły wejść (semafor blokował dostęp).
Rozwiązanie: Zdiagnozowano, że procesy "odrzucone" (które weszły na kładkę, zobaczyły zamkniętą trasę i zawróciły) kończyły działanie bez zwolnienia semafora rezerwującego miejsce. Dodano wywołanie `unlock_sem` w ścieżce wyjścia awaryjnego turysty.

3. Testy

- Przepustowość wąskiego gardła (Kładka)

Cel : Weryfikacja, czy limit osób na moście jest przestrzegany i czy zachowany jest ruch jednokierunkowy.

Przebieg: Uruchomiono symulację z procesami turystów. Obserwowano licznik `people_on_bridge` w panelu Przewodnika oraz logi systemowe.

Wynik:

- Licznik osób na moście nigdy nie przekroczył wartości `LIMIT_BRIDGE` (zdefiniowanej w `common.h` jako 3).
- W logach zaobserwowano sekwencje: grupa wchodzi w jednym kierunku -> most pustoszeje -> grupa wchodzi w przeciwnym kierunku. Potwierdza to poprawność logiki funkcji `czekaj_na_most` (plik `klient.cpp`), która blokuje dostęp przy niezgodnym kierunku ruchu.

- **Obsługa priorytetów**

Cel: Sprawdzenie, czy klienci powracający są obsługiwani poza kolejnością.

Przebieg: Uruchomiono symulację z procesami turystów, w której losowani są turyści z flagą `is_repeater = 1`

Wynik:

- Proces kasjer odebrał komunikat od klienta powracającego (typ wiadomości `mtype=2`) natychmiast, mimo że w kolejce czekały starsze komunikaty typu 1.

- **Restrykcje regulaminowe (Wiek i Trasy)**

Cel: Weryfikacja poprawności przydziału tras dla grup wiekowych (Dzieci <8 lat, Seniorzy >75 lat).

Przebieg: Analiza pliku `raport_kasjer.txt` pod kątem korelacji wieku z wybraną trasą.

Wynik:

- 100% przypadków turystów z wiekiem < 8 lat skutkowało przypisaniem do Trasy2
- 100% przypadków wieku > 75 lat skutkowało przypisaniem do Trasy 2.
- Potwierdza to działanie warunków logicznych w generatorze (`klient.cpp`), który wymusza `route=2` dla tych przedziałów.

- **Dynamiczne zamykanie i otwieranie tras (Strażnik)**

Cel: Sprawdzenie reakcji systemu na sygnały zewnętrzne oraz poprawności zwalniania zasobów po odmowie wejścia.

Przebieg:

1. Uruchomienie straznik i wysłanie sygnału zamknięcia Trasy 1 (`SIGUSR1`).
2. Obserwacja procesów próbujących wejść na Trasę 1.
3. Ponowne otwarcie Trasy 1 po chwili.

Wynik:

- Procesy Przewodnika zmieniły status na "ZAMKNIĘTA".
- Nowe procesy po wejściu na kładkę odczytały zamknięty status, wypisały log [!] Trasa ZAMKNIETA... Powrot i zakończyły działanie.
- Po ponownym otwarciu trasy, nowe procesy mogły swobodnie wchodzić – co potwierdza, że procesy "odrzucone" poprawnie zwolniły semafor

4. Instrukcja kompilacji i uruchomienia

Projekt wykorzystuje narzędzie make do automatyzacji procesu kompilacji wszystkich modułów jednocześnie.

Kompilacja:

W katalogu z plikami źródłowymi należy wywołać komendę: **make**

System automatycznie skompiluje pliki: main, kasjer, klient, przewodnik, straznik oraz cleanup, linkując je z biblioteką tools.cpp.

Uruchomienie symulacji:

System składa się z dwóch niezależnych okien terminala:

TERMINAL 1:

Symulacja (Manager) Uruchamia główny proces, który inicjuje środowisko, tworzy zasoby IPC i startuje procesy potomne (turystów, kasjera, przewodnika).

Należy wywołać komendę: **./main**

TERMINAL 2:

Sterowanie (Strażnik) Służy do wysyłania sygnałów sterujących (zamykanie tras). Program automatycznie pobiera PID procesów z pamięci dzielonej.

Należy wywołać komendę: **./straznik**

Zakończenie i czyszczenie

Aby poprawnie zakończyć symulację, w Terminalu 1 należy wcisnąć **Ctrl+C** (Manager posprząta procesy potomne).

Dodatkowe polecenia czyszczące:

Usuwanie zasobów IPC (pamięć/semafony) - należy wpisać komendę **./cleanup**

Usuwanie plików binarnych i logów: - należy wpisać komendę **make clean**

5. Implementacja - Linki do kodów

a) Tworzenie i obsługa procesów (fork, exit, wait, execlp)

- **fork()**
Wykorzystywana w module generatora ruchu (klient.cpp). Wewnątrz pętli głównej proces macierzysty wywołuje fork(), tworząc procesy potomne symulujące pojedynczych turystów. Każdy potomek otrzymuje kopię pamięci rodzica, ale wykonuje niezależną ścieżkę kodu (turist_life).
LINK: [fork\(\)](#)
- **exit()**
Służy do bezpiecznego zakończenia procesu turysty po zwiedzeniu jaskini (lub rezygnacji) i odłączeniu pamięci dzielonej. Zwraca kod wyjścia 0 do systemu.
LINK: [exit\(\)](#)
- **wait()**
Użyta w generatorze, aby oczekiwać na zakończenie procesów potomnych. Zapobiega to powstawaniu tzw. procesów zombie, które blokowałyby tablicę procesów w systemie.
LINK: [wait\(\)](#)
- **execlp()**
Wykorzystywana w pliku main.cpp tuż po operacji fork(). Funkcja ta podmienia obraz bieżącego procesu na nowy program (np. ./kasjer, ./klient). Dzięki temu proces potomny przestaje być kopią managera, a staje się niezależnym programem realizującym konkretne zadanie symulacji.
LINK: [execlp\(\)](#)

b) Obsługa sygnałów (signal, kill)

- **signal()**
Użyta w procesie przewodnik.cpp do zarejestrowania własnych funkcji obsługi dla sygnałów SIGUSR1 oraz SIGUSR2. Dzięki temu proces nie jest ubijany przez te sygnały, lecz wykonuje logikę zamknięcia/otwarcia trasy.
LINK: [signal\(\)](#)
- **kill()**
Wykorzystana w panelu sterowania (straznik.cpp). Pozwala użytkownikowi wysłać sygnał do procesu Przewodnika (o wskazanym PID), co symuluje interwencję obsługi technicznej jaskini.
LINK: [kill\(\)](#)

c) Segmenty pamięci dzielonej (shmget, shmat)

- **shmget()**
Służy do utworzenia segmentu pamięci współdzielonej w systemie, identyfikowanego unikalnym kluczem. Pamięć ta przechowuje strukturę CaveState ze stanem całej jaskini.
LINK: [shmget\(\)](#)
- **shmat()**
Przyłącza utworzony segment pamięci do przestrzeni adresowej procesu. Dzięki temu procesy Przewodnika, Kasjera i Klienta mogą operować na tych samych zmiennych (licznikach osób).
LINK: [shmat\(\)](#)

d) Synchronizacja procesów (semop, semget)

- **semop()**
Kluczowa funkcja systemowa służąca do operacji na semaforach. W projekcie została opakowana w funkcje pomocnicze lock_sem (operacja -1) i unlock_sem (operacja +1) w pliku tools.cpp. Służy do realizacji sekcji krytycznych (ochrona pamięci dzielonej) oraz wstrzymywania turystów przed kłódką.
LINK: [semop\(\)](#)

e) Kolejki komunikatów (msgsnd, msgrcv)

- **msgsnd()**
Wysyła strukturę z danymi turysty (wiek, typ biletu) do kolejki komunikatów. Używana przez proces klienta do "ustawienia się w kolejce" do kasy.
LINK: [msgsnd\(\)](#)
- **msgrcv()**
Odbiera komunikaty z kolejki. Zastosowanie flagi IPC_NOWAIT oraz selektywnego odbierania (typ komunikatu 2) pozwala Kasjerowi w pierwszej kolejności obsługiwać klientów priorytetowych (powracających), co realizuje wymóg obsługi priorytetów.
LINK: [msgrcv\(\)](#)

f) Tworzenie i obsługa plików

- **open()**

Funkcja używana w pliku main.cpp do otwarcia (lub utworzenia) plików tekstowych raport_kasjer.txt oraz raport_ruch.txt. Użyto flag O_CREAT (utwórz, jeśli nie istnieje) i O_TRUNC (wyczyść, jeśli istnieje), aby przygotować plik do zapisu logów

LINK: [open\(\)](#)

- **close()**

Używana w main.cpp do zamknięcia deskryptora pliku po wykonaniu przekierowania (dup2). Jest to konieczne dla zachowania higieny zasobów systemowych.

LINK: [close\(\)](#)

g) Łączy nazwane i nienazwane

- **dup2()**

Kluczowa funkcja użyta w main.cpp do przekierowania standardowego wyjścia (STDOUT_FILENO) procesów potomnych. Podmienia ona domyślne wyjście na ekran (terminal) na deskryptor pliku otwartego wcześniej przez open(). Dzięki temu wszystkie cout z procesów Kasjera i Klienta trafiają prosto do plików .txt

LINK: [dup2\(\)](#)